

Scientific Computing: Exercises

Thorsten Koch



Applied Algorithmic Intelligence Methods department at ZIB
Chair of *Software and Algorithms for Discrete Optimization*
at TU Berlin



The solution must be submitted by the following Thursday before the lecture.

Programming exercises must be submitted as follows:

- ▷ Put everything together in one .zip file. **This should be named:**
 - **exNN-MMMMMM-lastname-firstname.zip**
 - NN = 01 .. 17 number of the exercise
 - MMMMMM is your 6-digit Matrikelnummer (all who have none, send a message on ISIS and we assign you a number)
 - lastname is your Lastname (only use characters a-z in English alphabet)
 - firstname is your Firstname (only use characters a-z in English alphabet)
- ▷ The .zip file should extract to a folder of the same name (without the .zip extension).
- ▷ When we do cargo run ... in that folder, your program must produce the desired output.
- ▷ The programs should include comments to explain what they do,
- ▷ NO HARD CODED PATHS, no warnings from cargo clippy.
- ▷ In written exercises, the steps for solving the problem need to be explained appropriately.
- ▷ **We will ignore any submission that does not conform to the above.**
- ▷ **All exercises must be written in English.**

We will put up an PDF with the latest exercise on ISIS every Thursday after the lecture.

Written exercises have to be submitted the same way, only as an .pdf file instead of a .zip file.

L04

Complexity theory

1. Big- $\mathcal{O}$ notation

Find the time complexity of the different versions of the Fibonacci number computation given `fibonacci.rs` on ISIS with the slides from the lecture.

2. Sort the sequence of functions below: (i) assuming asymptotic growth, where $g \in \mathcal{O}(f)$ holds true for the functions g, f in a sorted sequence, and (ii) for $n = 74$ and give a short explanation.

- $4\sqrt{n}$
- $2n^3 + 4n^2 + 950$
- $\frac{n}{10000} \log_2 n$
- $\frac{1}{100} n!$
- $45 \log_2 n^2 + 9$
- 1
- $10^{110}n + 110$
- $10n + 3^n$

The Binary Knapsack problem is defined as:

$$\max_{x_i \in \{0,1\}^n} \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq C$$

(See also Roughgarden, Chapter 16.5, or https://en.wikipedia.org/wiki/Knapsack_problem, or https://www.youtube.com/watch?v=xOlhR_2QCXY)

Write a Rust program that computes the optimal solution for this problem. Use `struct` to structure your program. The problem data should be put in via the standard input. The output should be the ids of the chosen items, the total objective value of the knapsack, and the time needed.

```
$ cargo run <test1.ksp
```

```
Items: 1 3
```

```
Total: 12
```

```
Time : 0.03 s
```

Furthermore, compute the worst-case runtime of your code, i.e., the $\mathcal{O}(n)$. Please add this to the written exercise.

The files with the problem instances will have the extension “.ksp” and the following format:

The first line is the number: n .

Each of the n following lines describe an item and contains 3 integers: the id of the item: i ; the value of the item: v_i ; and the weight of the item w_i .

The last line contains an integer describing the knapsack capacity, C .

For example, if the file contains the following:

```
3
1 3 8
2 2 8
3 9 1
10
```

This describes a problem instance in which there are $n=3$ items and the knapsack has a capacity of $C=10$. The item with id 1 has a value of 3 and a weight of 8. The item with id 2 has a value of 2 and a weight of 8. The item with id 3 has a value of 9 and a weight of 1.

The optimal solution has value 12 using items 1 and 3.

```
use std::time::Instant;

let start_time = Instant::now();
// Do something here
let duration    = start_time.elapsed();

println!("Time : {:?}", duration);
println!("Time : {:5}", duration.as_millis());
```

Formatted printing: `{:12.3e}`

See <https://doc.rust-lang.org/std/fmt>